

07_tarea_grafico_lineas_dobles_avocado

April 27, 2026

1 Tarea paso a paso: Gráfico de Líneas con Doble Eje Y

¡ATENCIÓN! Lee esto antes de empezar: Esta tarea está diseñada paso a paso para que afiances los conceptos vistos en clase sobre los **Dual-Axis Line Charts**. Su objetivo es que practiques cómo representar **dos series temporales con escalas muy distintas** en un único gráfico compartiendo el mismo eje X.

Nota importante sobre la Inteligencia Artificial: Es tentador copiar y pegar el enunciado de estos ejercicios en ChatGPT o Claude para que te den la solución... **Hacer eso perjudicará gravemente tu aprendizaje. (CODEA UN POCO QUE TE ENTERES QUE HACES).** La única forma de asimilar la sintaxis de las herramientas de visualización es escribiéndolas. **Usa tu cerebro, actúa paso a paso, equivócate, lee la documentación de Matplotlib y Plotly, busca tu fallo y ¡aprende de él!**

1.1 Preparación y Datos

Para esta tarea vamos a utilizar el dataset “**Avocado Prices**”, que recoge el precio medio semanal y el volumen total de aguacates vendidos en distintas ciudades de Estados Unidos entre 2015 y 2018.

Es un dataset ideal para un **Dual-Axis Line Chart** porque las dos variables de interés tienen escalas completamente distintas: - **AveragePrice**: precio medio de un aguacate, en dólares (~0.5 \$ a ~3 \$). - **Total Volume**: número total de aguacates vendidos (~1.000 a ~60.000.000).

Si las pusiésemos en el mismo eje Y, la línea del precio quedaría completamente aplastada y sería ilegible.

1. Descarga el dataset de Kaggle: [Avocado Prices](#)
2. Guarda el archivo `avocado.csv` en la carpeta `data/` de tu proyecto.
3. En la siguiente celda, importa las librerías necesarias y carga el CSV en un DataFrame llamado `df`.

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import plotly.graph_objects as go
from mpl_toolkits.axes_grid1 import host_subplot

# CARGA DE DATOS
df = pd.read_csv("data/avocado.csv")
```

```
df.head(5)
```

```
[1]: Unnamed: 0      Date AveragePrice Total Volume      4046      4225 \
0      0  2015-12-27      1.33      64236.62  1036.74  54454.85
1      1  2015-12-20      1.35      54876.98   674.28  44638.81
2      2  2015-12-13      0.93     118220.22   794.70 109149.67
3      3  2015-12-06      1.08      78992.15  1132.00   71976.41
4      4  2015-11-29      1.28      51039.60   941.48   43838.39
```

```
      4770 Total Bags Small Bags Large Bags XLarge Bags      type \
0  48.16    8696.87    8603.62      93.25         0.0 conventional
1  58.33    9505.56    9408.07      97.49         0.0 conventional
2 130.50    8145.35    8042.21     103.14         0.0 conventional
3  72.58    5811.16    5677.40     133.76         0.0 conventional
4  75.78    6183.95    5986.26     197.69         0.0 conventional
```

```
      year region
0  2015  Albany
1  2015  Albany
2  2015  Albany
3  2015  Albany
4  2015  Albany
```

```
[2]: # Ver la información del dataset
df.describe(include="all")
```

```
[2]: Unnamed: 0      Date AveragePrice Total Volume      4046 \
count  18249.000000    18249  18249.000000  1.824900e+04  1.824900e+04
unique          NaN      169          NaN          NaN          NaN
top          NaN  2015-12-27          NaN          NaN          NaN
freq          NaN      108          NaN          NaN          NaN
mean      24.232232      NaN      1.405978  8.506440e+05  2.930084e+05
std      15.481045      NaN      0.402677  3.453545e+06  1.264989e+06
min         0.000000      NaN      0.440000  8.456000e+01  0.000000e+00
25%      10.000000      NaN      1.100000  1.083858e+04  8.540700e+02
50%      24.000000      NaN      1.370000  1.073768e+05  8.645300e+03
75%      38.000000      NaN      1.660000  4.329623e+05  1.110202e+05
max      52.000000      NaN      3.250000  6.250565e+07  2.274362e+07
```

```
      4225      4770 Total Bags Small Bags Large Bags \
count  1.824900e+04  1.824900e+04  1.824900e+04  1.824900e+04  1.824900e+04
unique          NaN          NaN          NaN          NaN          NaN
top          NaN          NaN          NaN          NaN          NaN
freq          NaN          NaN          NaN          NaN          NaN
mean  2.951546e+05  2.283974e+04  2.396392e+05  1.821947e+05  5.433809e+04
std  1.204120e+06  1.074641e+05  9.862424e+05  7.461785e+05  2.439660e+05
```

min	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00
25%	3.008780e+03	0.000000e+00	5.088640e+03	2.849420e+03	1.274700e+02
50%	2.906102e+04	1.849900e+02	3.974383e+04	2.636282e+04	2.647710e+03
75%	1.502069e+05	6.243420e+03	1.107834e+05	8.333767e+04	2.202925e+04
max	2.047057e+07	2.546439e+06	1.937313e+07	1.338459e+07	5.719097e+06

	XLarge Bags	type	year	region
count	18249.000000	18249	18249.000000	18249
unique	NaN	2	NaN	54
top	NaN	conventional	NaN	Albany
freq	NaN	9126	NaN	338
mean	3106.426507	NaN	2016.147899	NaN
std	17692.894652	NaN	0.939938	NaN
min	0.000000	NaN	2015.000000	NaN
25%	0.000000	NaN	2015.000000	NaN
50%	0.000000	NaN	2016.000000	NaN
75%	132.500000	NaN	2017.000000	NaN
max	551693.650000	NaN	2018.000000	NaN

```
[3]: # Limpieza de nulos
df = df.dropna()
df
```

```
[3]:      Unnamed: 0      Date  AveragePrice  Total Volume      4046      4225  \
0              0  2015-12-27           1.33      64236.62  1036.74  54454.85
1              1  2015-12-20           1.35      54876.98   674.28  44638.81
2              2  2015-12-13           0.93     118220.22   794.70  109149.67
3              3  2015-12-06           1.08      78992.15  1132.00   71976.41
4              4  2015-11-29           1.28      51039.60   941.48   43838.39
...           ...      ...      ...      ...      ...      ...
18244          7  2018-02-04           1.63      17074.83  2046.96   1529.20
18245          8  2018-01-28           1.71      13888.04  1191.70   3431.50
18246          9  2018-01-21           1.87      13766.76  1191.92   2452.79
18247         10  2018-01-14           1.93      16205.22  1527.63   2981.04
18248         11  2018-01-07           1.62      17489.58  2894.77   2356.13
```

	4770	Total Bags	Small Bags	Large Bags	XLarge Bags	type
0	48.16	8696.87	8603.62	93.25	0.0	conventional
1	58.33	9505.56	9408.07	97.49	0.0	conventional
2	130.50	8145.35	8042.21	103.14	0.0	conventional
3	72.58	5811.16	5677.40	133.76	0.0	conventional
4	75.78	6183.95	5986.26	197.69	0.0	conventional
...	...	...	...	...	...	...
18244	0.00	13498.67	13066.82	431.85	0.0	organic
18245	0.00	9264.84	8940.04	324.80	0.0	organic
18246	727.94	9394.11	9351.80	42.31	0.0	organic
18247	727.01	10969.54	10919.54	50.00	0.0	organic

```
18248  224.53      12014.15    11988.14      26.01      0.0      organic
```

```

      year      region
0    2015      Albany
1    2015      Albany
2    2015      Albany
3    2015      Albany
4    2015      Albany
...
18244  2018  WestTexNewMexico
18245  2018  WestTexNewMexico
18246  2018  WestTexNewMexico
18247  2018  WestTexNewMexico
18248  2018  WestTexNewMexico

```

```
[18249 rows x 14 columns]
```

1.2 Paso 1: Filtrar y Preparar los Datos

Objetivo: Obtener una serie temporal limpia filtrando el dataset a un único mercado y tipo de aguacate, y agrupando por año.

Instrucciones: 1. Filtra el DataFrame para quedarte solo con las filas donde `region == 'TotalUS'` y `type == 'conventional'`. Guarda el resultado en `df_us`. 2. Agrupa `df_us` por la columna `year` y calcula: - La **media** de `AveragePrice`. - La **suma** de `Total Volume`.

Usa `.agg({'AveragePrice': 'mean', 'Total Volume': 'sum'})` y guarda en `df_agrupado`. 3. Muestra `df_agrupado` para verificar que tienes 4 filas (una por año: 2015, 2016, 2017, 2018).

```
[4]: # filtramos region TotalUS y aguacate convencional, agrupamos por año
      # mean en precio porque se promedia a lo largo del año, sum en volumen porque
      ↪ se acumula
df_us = df[(df['region'] == 'TotalUS') & (df['type'] == 'conventional')]
df_agrupado = df_us.groupby('year').agg({
    'AveragePrice': 'mean',
    'Total Volume': 'sum'
})
df_agrupado
```

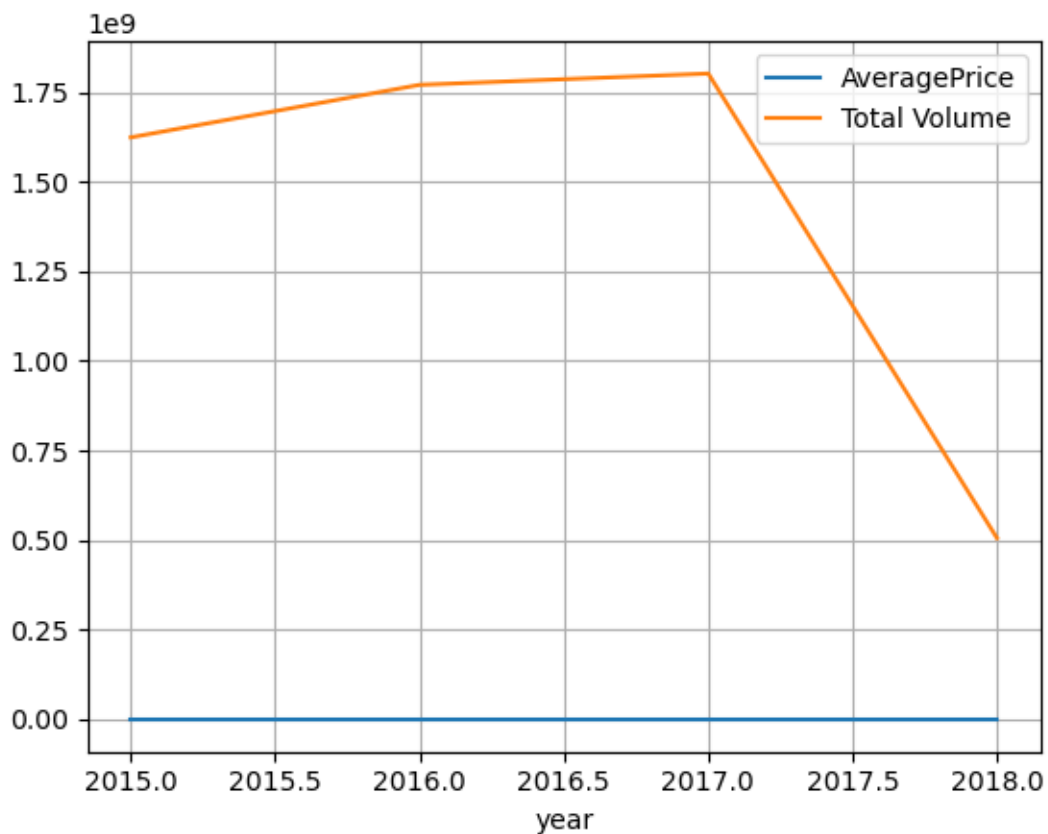
```
[4]:      AveragePrice  Total Volume
year
2015      1.012500  1.623686e+09
2016      1.046731  1.770259e+09
2017      1.221698  1.801770e+09
2018      1.060000  5.055064e+08
```

1.3 Paso 2: El Problema del Eje Único

Objetivo: Ver con tus propios ojos por qué necesitamos un doble eje Y cuando las variables tienen escalas muy distintas.

Instrucciones: 1. Usando `df_agrupado`, dibuja un gráfico de líneas con `df_agrupado.plot.line()` que muestre **las dos variables** (`AveragePrice` y `Total Volume`) en el **mismo eje Y**. Usa el parámetro `y=['AveragePrice', 'Total Volume']`. 2. Activa la cuadrícula con `grid=True`. 3. Muestra el gráfico. ¿Qué le ocurre a la línea de `AveragePrice`? ¿Por qué?

```
[5]: # Vemos como se queda si graficamos todo en un mismo eje, perdemos noción de la
      ↪escala
df_agrupado.plot.line(
    y=['AveragePrice', 'Total Volume'],
    grid=True
)
plt.show()
```



1.3.1 Interpretación

La línea de `AveragePrice` queda completamente aplastada contra el eje X porque sus valores (entre 1 y 2) son insignificantes comparados con los de `Total Volume` (en el orden de los miles de millones). Necesitamos un segundo eje Y con su propia escala.

1.4 Paso 3: Doble Eje Y con Matplotlib (`twinx`)

Objetivo: Crear el gráfico de doble eje Y usando el método `twinx()` de Matplotlib, que crea un segundo eje Y que comparte el mismo eje X.

Instrucciones: 1. Crea la figura con `fig, ax1 = plt.subplots(figsize=(10, 5))`. 2. Dibuja la primera línea en `ax1`: usa `ax1.plot(...)` con el índice de `df_agrupado` como X y `df_agrupado['AveragePrice']` como Y. Ponle un color (por ejemplo, `'tab:green'`) y una etiqueta. 3. Crea el segundo eje con `ax2 = ax1.twinx()`. Dibuja la segunda línea en `ax2` con `ax2.plot(...)` usando `df_agrupado['Total Volume']` como Y y un color distinto (por ejemplo, `'tab:orange'`). 4. Etiqueta los ejes Y de cada eje con `ax1.set_ylabel(...)` y `ax2.set_ylabel(...)`, usando el mismo color que su línea para que quede claro cuál eje corresponde a qué variable. 5. Añade un título con `plt.title(...)` y muestra el gráfico.

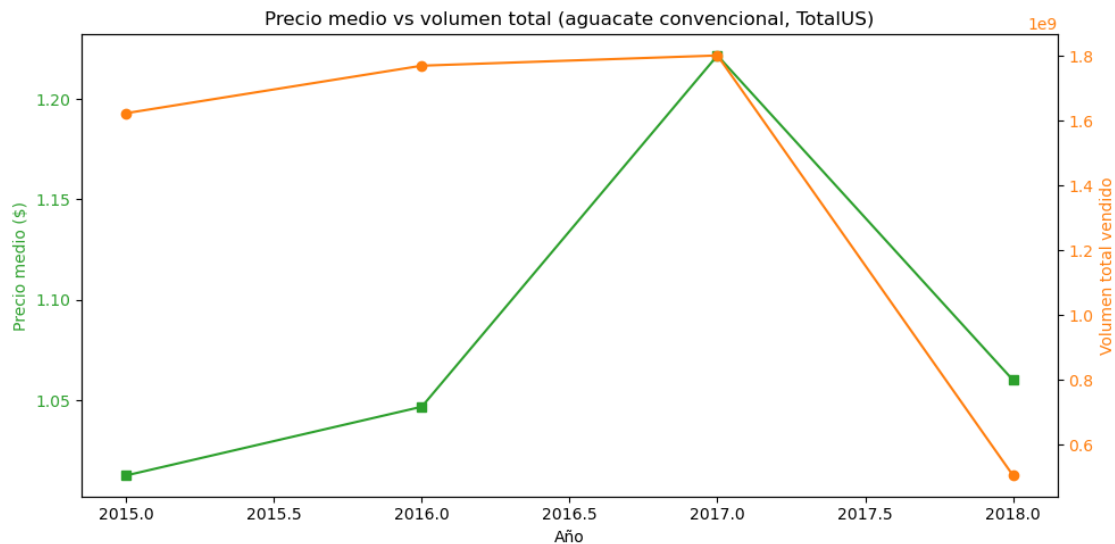
```
[6]: # Creación de un gráfico con dos ejes Y utilizando Matplotlib
# https://matplotlib.org/stable/gallery/subplots_axes_and_figures/two_scales.
# ↪html
fig, ax1 = plt.subplots(figsize=(10, 5))

ax1.set_xlabel('Año')
ax1.set_ylabel('Precio medio ($)', color='tab:green') # verde por aguacate
ax1.plot(
    df_agrupado.index.values,
    df_agrupado['AveragePrice'],
    color='tab:green',
    marker='s'
)
ax1.tick_params(axis='y', labelcolor='tab:green')

# Segundo eje Y compartiendo X
ax2 = ax1.twinx()
ax2.set_ylabel('Volumen total vendido', color='tab:orange') # naranja para
# ↪distinguir
ax2.plot(
    df_agrupado.index.values,
    df_agrupado['Total Volume'],
    color='tab:orange',
    marker='o'
)
ax2.tick_params(axis='y', labelcolor='tab:orange')

plt.title('Precio medio vs volumen total (aguacate convencional, TotalUS)')
fig.tight_layout()
```

```
plt.show()
```



1.5 Paso 4: Doble Eje Y con Plotly

Objetivo: Reproducir el mismo gráfico de doble eje Y usando Plotly, que ofrece interactividad (zoom, hover, toggle de series) sin esfuerzo extra.

Instrucciones: 1. Crea una figura vacía con `fig = go.Figure()`. 2. Añade la primera serie con `fig.add_trace(go.Scatter(...))`. Usa el índice de `df_agrupado` como `x`, `df_agrupado['AveragePrice']` como `y`, `name='Precio Medio ($)'`, `mode='markers+lines'` y `marker=dict(color='green')`. 3. Añade la segunda serie con otro `fig.add_trace(go.Scatter(...))`. Esta vez añade `yaxis='y2'` para indicar que usa el eje Y secundario. Usa un color distinto. 4. Configura el layout con `fig.update_layout(...)` incluyendo: - `title`: un título descriptivo. - `yaxis`: diccionario con `title='Precio Medio ($)'`. - `yaxis2`: diccionario con `title='Volumen Total de Ventas'`, `overlaying='y'` y `side='right'`. 5. Muestra el gráfico con `fig.show()`.

```
[7]: # Grafico de lineas con doble eje Y en Plotly
fig = go.Figure()

# Primera serie: precio en el eje Y principal
fig.add_trace(
    go.Scatter(
        x=df_agrupado.index.values,
        y=df_agrupado['AveragePrice'],
        name='Precio Medio ($)',
        marker=dict(color='green'),
        mode='markers+lines'
    )
)
```

```

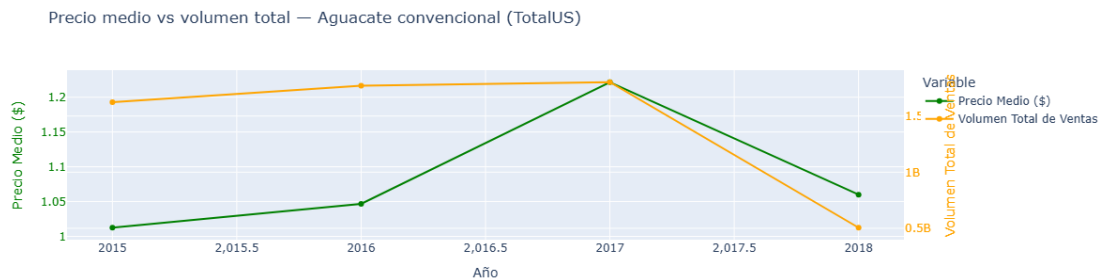
)

# Segunda serie: volumen en el eje Y secundario (yaxis='y2' es la clave)
fig.add_trace(
    go.Scatter(
        x=df_agrupado.index.values,
        y=df_agrupado['Total Volume'],
        name='Volumen Total de Ventas',
        marker=dict(color='orange'),
        mode='markers+lines',
        yaxis='y2'
    )
)

fig.update_layout(
    title='Precio medio vs volumen total - Aguacate convencional (TotalUS)',
    xaxis_title='Año',
    yaxis=dict(
        title=dict(text='Precio Medio ($)', font=dict(color='green')),
        tickfont=dict(color='green')
    ),
    yaxis2=dict(
        title=dict(text='Volumen Total de Ventas', font=dict(color='orange')),
        tickfont=dict(color='orange'),
        overlaying='y',          # superponer al eje Y principal
        side='right'             # colocar el eje secundario a la derecha
    ),
    legend_title='Variable'
)

fig.show()

```



1.5.1 Interpretación del Gráfico de Doble Eje

Al usar un **doble eje Y**, cada variable tiene su propia escala y ambas líneas son perfectamente legibles. Ahora podemos extraer conclusiones reales:

1. **Eje Y izquierdo (Precio):** muestra la evolución del precio medio del aguacate en dólares a lo largo de los años.
2. **Eje Y derecho (Volumen):** muestra el total de aguacates vendidos en EE. UU., una magnitud millones de veces mayor.
3. **Conclusión:** ¿Sube el precio cuando baja el volumen vendido, o viceversa? El gráfico te permite responder esta pregunta de un vistazo, algo imposible con un eje único.

1.6 Reflexión Final del Alumno

Tómate un rato para reflexionar. * El **gráfico de doble eje Y** resuelve un problema muy concreto: comparar la evolución temporal de dos variables con escalas incompatibles. Sin él, una de las dos series quedaría invisible o distorsionada. * Recuerda su principal riesgo: este tipo de gráfico puede ser **engañoso** si no se etiquetan correctamente los ejes o si se usan para sugerir correlaciones que no existen. Úsalo con honestidad y siempre indica claramente qué eje corresponde a qué variable. * Como reto adicional: ¿podrías repetir el análisis pero filtrando por `type == 'organic'`? ¿El comportamiento del precio y el volumen es diferente al de los aguacates convencionales?

Asegúrate de que cada celda tenga sus respectivos comentarios.

El precio se dispara y el volumen se hunde, el orgánico es un nicho. La forma de la curva yo creo que es la misma pero a otra escala.

```
[8]: # Reto: mismo análisis con aguacate orgánico (espero precio más alto, volumen
      mucho menor)
df_us_org = df[(df['region'] == 'TotalUS') & (df['type'] == 'organic')]
df_agrupado_org = df_us_org.groupby('year').agg({
    'AveragePrice': 'mean',
    'Total Volume': 'sum'
})

fig, ax1 = plt.subplots(figsize=(10, 5))

ax1.set_xlabel('Año')
ax1.set_ylabel('Precio medio ($)', color='tab:green')
ax1.plot(
    df_agrupado_org.index.values,
    df_agrupado_org['AveragePrice'],
    color='tab:green',
    marker='s'
)
ax1.tick_params(axis='y', labelcolor='tab:green')

ax2 = ax1.twinx()
ax2.set_ylabel('Volumen total vendido', color='tab:orange')
ax2.plot(
```

```

df_agrupado_org.index.values,
df_agrupado_org['Total Volume'],
color='tab:orange',
marker='o'
)
ax2.tick_params(axis='y', labelcolor='tab:orange')

plt.title('Precio medio vs volumen total (aguacate orgánico, TotalUS)')
fig.tight_layout()
plt.show()

```

